



When I first started working with APIs in React, my code looked something like this.

```
axios.get("/users", {  
  headers: {  
    Authorization: `Bearer ${token}`,  
  },  
});
```

Then another API.

```
axios.get("/products", {  
  headers: {  
    Authorization: `Bearer ${token}`,  
  },  
});
```

```
    },  
  });
```

Then another.

```
axios.post("/orders", orderData, {  
  headers: {  
    Authorization: `Bearer ${token}`,  
  },  
});
```

At first, it didn't seem like a big problem.

But as the project grew, I realized I was writing the same code over and over again.

Every request needed an authentication token.

Every response needed error handling.

Every failed request needed similar logic.

That's when I came across one of the most useful features in Axios:

Interceptors.

. . .

What Are Axios Interceptors?

In simple words,

Axios Interceptors allow us to intercept every HTTP request before it is sent and every HTTP response before it reaches our application.

Think of them as checkpoints.

Instead of allowing every request to directly reach the server, Axios gives us an opportunity to inspect or modify it first.

Similarly, when the server sends back a response, we get another opportunity to inspect or modify it before our React application receives it.

The complete flow looks like this:

React Application

↓

Request Interceptor

↓

Axios

↓

Server

↓

Response Interceptor

↓

Notice something interesting.

Our application never talks directly to the server.

Every request and every response passes through these interceptors.

. . .

Before Understanding Interceptors, Let's Understand Middleware

Whenever I explain interceptors, I always explain middleware first because the idea is exactly the same.

Imagine you're entering an office building.

You don't directly enter the office.

First,

You

↓

Security Check

↓

Reception

↓

Office

The security guard checks your ID.

The receptionist verifies your appointment.

Only then are you allowed to enter.

The security guard isn't the office.

The receptionist isn't the office.

They're simply stopping you in the middle to perform some additional work.

That's exactly what middleware is.

A middleware sits **between two systems** and performs some work before allowing the request to continue.

Now replace the office with a server.

React App

↓

Middleware (Interceptor)

↓

Server

Axios Interceptors are nothing but middleware for your HTTP requests and responses.

. . .

How Do Interceptors Work?

Axios provides two types of interceptors.

1. Request Interceptor

This runs before the request is sent to the server.

Component

↓

Request Interceptor

↓

Server

Here we can:

- Add authentication tokens
- Add custom headers
- Modify request data

- Log outgoing requests

Once we're done, the request continues to the server.

• • •

2. Response Interceptor

This runs after the server responds but before the response reaches your application.

Server

↓

Response Interceptor

↓

Component

Here we can:

- Handle errors
- Transform response data
- Log responses
- Retry failed requests
- Redirect users

. . .

Behind the Scenes

Suppose we write:

```
axios.get("/users");
```

It may look like Axios directly sends the request.



But internally, something like this happens.

```
axios.get("/users")
```

↓

Request Interceptor

↓

HTTP Request

↓

Server

↓

HTTP Response

↓

Response Interceptor

↓

.then()

↓

Your Component

This is why interceptors are so powerful.

They allow us to execute common logic without touching individual API calls.

. . .

Why Do We Need Interceptors?

Let's imagine a project with 100 API calls.

Without interceptors, every request may look like this.

```
axios.get("/users", {  
  headers: {  
    Authorization: `Bearer ${token}`,  
  },  
});
```

```
axios.get("/products", {  
  headers: {  
    Authorization: `Bearer ${token}`,  
  },  
});
```

```
    },  
  });
```

```
axios.post("/orders", data, {  
  headers: {  
    Authorization: `Bearer ${token}`,  
  },  
});
```

The same code is repeated everywhere.

Now imagine the authentication format changes tomorrow.

Instead of changing one file,

you'll have to update every API call.

That's not scalable.

Interceptors solve this problem by centralizing common logic.

Instead of repeating code,

we write it once,

and Axios automatically applies it to every request or response.

. . .

Industry Example #1 — Automatically Attaching JWT Tokens

This is probably the most common use case.

Suppose your application stores an access token after login.

Every protected API requires this token.

Without interceptors,

you'll manually attach the token everywhere.

With a Request Interceptor,

the process becomes automatic.

API Call

↓

Request Interceptor

↓

Authorization Header Added

↓

Server

Now every request carries the token without writing the same code again and again.

This is exactly how many production applications handle authentication.

. . .

Industry Example #2 — Global 401 Error Handling

Imagine the user's access token expires.

The server responds with:

```
401 Unauthorized
```

Without interceptors,

every API call needs its own error handling.

```
API 1
```

```
↓
```

```
401 Handling
```

```
API 2
```

```
↓
```

```
401 Handling
```

```
API 3
```

```
↓
```

```
401 Handling
```

The same logic gets duplicated across the project.

Instead, a Response Interceptor can catch every `401 Unauthorized` response in one place.

The application can then:

`401` Received

↓

Refresh Access Token

↓

Retry Original Request

↓

Return Successful Response

The user doesn't even realize that the token expired.

Everything happens behind the scenes.

This is one of the most common authentication flows used in modern web applications.

. . .

Should Every API Use Interceptors?

Not necessarily.

If you're building a very small project with only two or three API calls, using interceptors may not provide much benefit.

But once your application grows, you'll likely have dozens or even hundreds of requests.

At that point, interceptors become one of the best ways to keep your code clean, maintainable, and consistent.

. . .

Final Thoughts

When I first learned Axios, I thought it was just a library for making HTTP requests.

Later, I realized that making requests is only a small part of the story.

Real-world applications also need authentication, centralized error handling, request logging, response transformation, token refresh mechanisms, and many other cross-cutting concerns.

Axios Interceptors solve these problems elegantly by allowing us to intercept every request and response before they reach their destination.

If there's one thing you should remember from this article, let it be this:

Axios sends requests. Interceptors make those requests smarter.

Once you understand this idea, you'll start seeing why almost every production-grade React application uses Axios Interceptors.

Happy coding ❤️

- API +
- Communication +
- JavaScript ✓
- Programming ✓
- Web Development ✓



Written by Devendra Dhote

Follow

343 followers · 8 following

MERN Stack Developer & Instructor. Writing about JavaScript, React, Next.js, backend development, and the engineering concepts behind modern web applications.

[Placeholder text block]

[Placeholder text block]